3 The server SSL/TLS scanner

As discussed in the previous chapter, currently does not exist the tool, that precisely satisfies the introduced needs. In this chapter, we are going to discuss the development of the tool, that will be referred as the *Server scanner* in the rest of this thesis.

3.1 The functional requirements

First at all, we need to gather all the requirements:

- The support for the most of the tests, that were discussed in the previous chapter.
- The simple configuration, that validates the user input and warns about the typos.
- The easily interpretable output.
- The application should be easily extended.
- Runnable on the Linux operating systems.
- Suitable for the automated testing.

3.2 The software design

Rather than developing the Server scanner from the scratch, we will base it on the already existing tool. Implementation of all the required tests would be time-consuming and from the certain perspective useless. According to the previous chapter, the most promising looks O-Saft by the OWASP. It supports the wide range of the scans and comes with the ability to list all the ciphers suites supported by a remote server. It does not have own API, but the output of O-Saft is ready to be parsed by the Server scanner.

3.2.1 The configuration

Because support for the automated testing is must, all the parameters for the Server scanner must be easily passable. From this reason, the Server scanner requires the two XML files, that altogether describes the behavior during the testing from startup to its quit.

The configuration file targets.xml

The following XML file tells the Server scanner, which the targets should be tested, how should be testing done and which the result is considered secure (or not).

Now, let's take a look at the example of such the file and then discuss some decisions, that led to its final form.

Code 3.1: The content of the file targets.xml

```
<configuration>
 <profiles>
        <!-- content of tag profiles -->
 </profiles>

 <targets>
   <!-- content of tag targets -->
 </targets>
</configuration>
```

The content of the targets.xml consists of the two main parts. Firstly the *testing profiles* are presented, then the *targets*, that will be scanned.

Code 3.2: The example content of the tag profiles

```
<profile name="low">
 <protocols>
  <protocol name="SSLv2" mode="mustNotBe" />
  <!-- shortened -->
 </protocols>
 <vulnerabilitiesFree mode="canBe" />
 <certificateValid  mode="canBe">
  <directive name="rsaMinimumPublicKeySize"
             value="2048"  mode="canBe" />
  <!-- shortened -->
 </certificateValid>
 <ciphers>
  <cipher name="ECDHE-ECDSA-DES-CBC3-SHA" mode="mustNotBe" />
```

```
  <!-- shortened -->
 </ciphers>
</profile>
```

Code 3.3: The example content of the tag targets

```
<target destination="https://domain.tld/"
        profile="low"
        name="name␣of␣target" />
```

Each profile comes with the name and also further clarifies the four main categories of the tests:

- *protocols* – Defines, which the protocols are considered secure.
- *vulnerabilities* – Defines, if the tests for the known vulnerabilities will be performed.
- *certificate* – Defines, if the tests for the attributes of the certificate will be performed. A user is required to state, which the size of the certificate's key is considered secure (in the case of RSA and ECDSA). There is also ability to define the user's (custom) certificate authority, that will be considered as trusted and used to test validity of the offered certificate.
- *ciphers* – Precisely defines, which the cipher suites are considered secure.

It is noteworthy, that the XML file requires explicit acknowledgment of the test result, that will be considered secure. This is accomplished by the attribute *mode*, which allows only the three values *canBe*, *mustBe*, *mustNotBe* (with the apparent meaning). The Server scanner is very strict during the parsing XML file. The missing and indwelling tags or attributes are reported as the error. The same applies for the range of allowed values of the selected attributes.

All these measures minimize the risk of the false secure test result, that was achieved by the unwanted user mistake in the configuration.

3.2.2 The implementation

To ensure, that the Server scanner will correctly run on the different Linux distributions, Java was chosen as the implementation language.

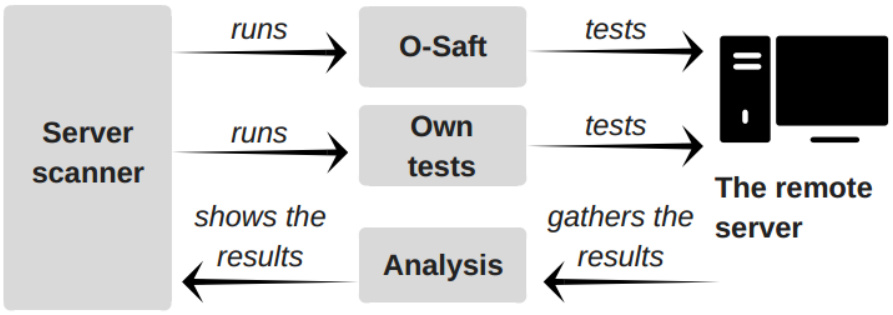


Figure 3.1: The simplified schema of the Server scanner approach

To run the Server scanner, the two prerequisites must by fulfilled – the presence of the correctly configured tool O-Saft and the Java Runtime Environment. No more third-party libraries are required. The user manual for the Server scanner is available in the appendix of this thesis.

3.2.3 The facade pattern

O-Saft is originally the command line tool. To make it easier to use in the environment of the Server scanner, the facade software design pattern was implemented. Because the Server scanner is based on object-oriented programming, the same object-oriented interface was required for O-Saft, and was achieved by the mentioned facade pattern. The class *OSaftFacade* responsible for the communication with O-Saft translates the requirements for the future tests to the command line arguments and runs O-Saft with them. Output is then captured and parsed with the help of the dedicated class named *OSaftParser*. Test results are then presented via the getter methods.

The Server scanner is ready for the possible future adjustments, that may involve the implementation of another tool (via its facade). The class *BaseFacade* should be extended and the parser for the output of the newly implemented tool must be written.

3.2.4 The test against custom certificate authority

In the case of any SSL/TLS enabled server, each certificate (offered by the server during the initial handshake) was always issued by *some* certificate authority. With the access to the root certificate of the selected authority, the offered certificate can be tested for its validation. The mentioned test is successful only if the offered certificate was genuinely issued by the certificate authority. The described feature was expressly requested during the development by the Y SOFT company to ensure the cooperation of the Server scanner with the Y SOFT's custom certificate authority. This test was implemented from the scratch, because O-Saft tool does not support this feature.

3.2.5 The different interpretation of O-Saft's output

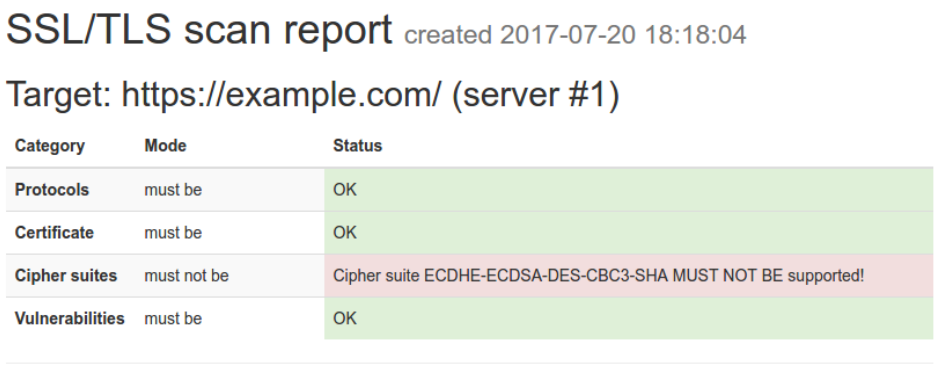
After the mutual agreement with the Y SOFT company, interpretation of the few O-Saft's tests was altered. Some practices, commonly used in the real production environment, O-Saft considers as unsafe. One of the possible examples may be the certificate, that uses the wildcard in its Common name (CN) attribute. The mentioned certificate is usually being referred as the *wildcard certificate*.

The wildcard certificate can be used across multiple domains or subdomains, which is usually more cheaper than buying the dedicated certificate for each required usage. The single wildcard certificate is also more convenient for the administration (for example during the renewal process, when only single certificate must be renewed). From the mentioned reasons, wildcard certificate is being offered by various certificate authorities and used in the production environment.

O-Saft considers wildcard certificate as unsafe, because such the certificate does not contain a complete list of all allowed domains or subdomains (instead wildcard represented by the asterisk symbol is used). Therefore, this evaluation was overridden in the Server scanner and correct usage of the wildcard certificate is not raising security notice.

3.2.6 The Gathering of the results

When the test results are available, it's needed to decide, which is considered unsafe. In the most of the cases, the mentioned decision was



SSL/TLS scan report created 2017-07-20 18:18:04

Target: https://example.com/ (server #1)

Category	Mode	Status
Protocols	must be	OK
Certificate	must be	OK
Cipher suites	must not be	Cipher suite ECDHE-ECDSA-DES-CBC3-SHA MUST NOT BE supported!
Vulnerabilities	must be	OK

Figure 3.2: The example output of the Server scanner

already made directly by O-Saft. But this is not the rule, for example in the case of the certificate's minimum key size, the necessary value (key size) is detected by O-Saft and final judgment is made by the Server scanner. Such the decisions are being made in the class *Scanner*.

3.2.7 Export to the HTML file

After gathering all the necessary information, output in the HTML form is created. Responsibility for such the task takes over the class *HtmlExport*, that is dedicated to the creation of the HTML file from the collection of gathered test results.

3.3 The support for continuous integration

The Server scanner can be integrated into a software development, that is based on the continuous integration. Because the testing is a key part of the continuous integration, the automated tests are appreciated thanks to their ability to minimize the time spent by the testing. In the real production environment, automated tests are usually handled by the specialized software intended for continuous integration, deployment and release management. The Server scanner is prepared for such software because its use can be automated. Firstly, two configuration files must be prepared, which is a one-time-only task. Then, the

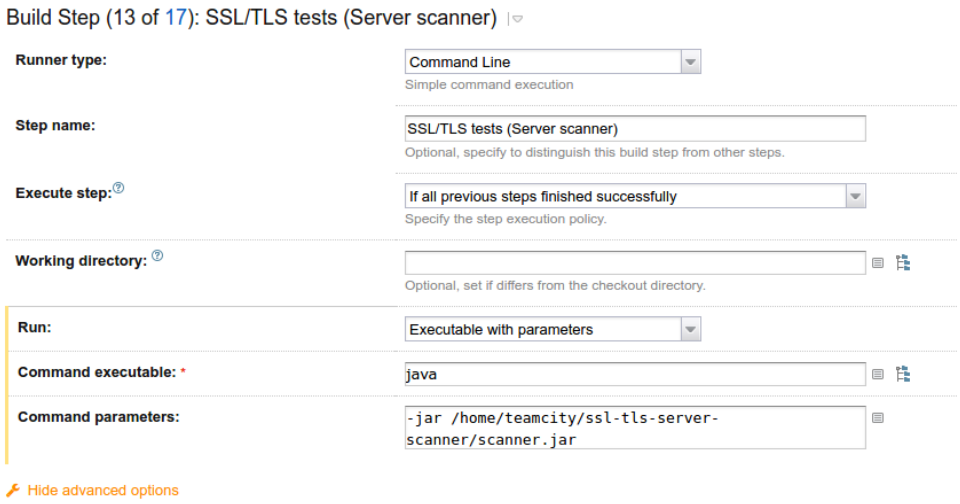


Figure 3.3: The example usage of the Server scanner with TeamCity (one of the available software solutions for continuous integration).

Server scanner can be automatically and repeatedly executed without any additional parameters.

The Server scanner always returns a boolean value indicating, if all test successfully passed or not. Such a boolean value is typically used by the software for continuous integration, that refuses to continue in the deployment if some tests are failing. In that case, a developer can look into the detailed output, that is created during every usage of the Server scanner.

3.4 The testing

To validate the Server scanner output, a new testing server was created. Its configuration was gradually altered in that ways, that all the tests performed by O-Saft and its interpretation by the Server scanner could be verified. Each configuration allowed some attacks to be performed. The Server scanner passed if this situation was correctly detected and reported to a user as the discovered vulnerability.

To simulate the vulnerable environment, the conditions described in the section 2.1 as unsafe was intentionally created. Let's take a look at the example, the FREAK is vulnerability over the RSA_EXPORT cipher suites. To verify the Server scanner's output, exactly such the RSA_EXPORT cipher suites was enabled on the testing server and the Server scanner was used to test this unsafe environment. The same logic applies to the remaining tests for vulnerabilities.

The opposite approach was needed for the conditions that strengthen the security of the system. For example, the HPKP (HTTP Public Key Pinning) is the mechanism with the desirable presence. The HPKP was allowed on the testing server and the Server scanner passed the testing, if the HPKP support was correctly discovered.

Altogether, according to the table 2.3, more than 30 tests were conducted to verify the Server scanner behavior.

During the testing was discovered the confusing O-Saft's text output regarding scans of the certificate. O-Saft interprets the result of each scan with the boolean value and the text note, that further describes the discovered vulnerability. In this case, the boolean value was correct, but the note contained text, that looked like the description of the different test result, then the boolean value implied. The mentioned issue [32] was reported to the author of O-Saft and resolved by him.

Another part of the testing was focused on the implementation of the Server scanner. Firstly, parsing of the configuration XML files was tested with the invalid content (for example the wrong format, the missing or the indwelling values, the unknown configuration directives). Then, the communication with O-Saft was verified. The tests included the situations, when O-Saft is not even located on the local machine or the unexpected value was returned by O-Saft. Lastly, the correct creation of the HTML file with the test results was tested. It was needed to check, that the results are printed in the correct categories and the vulnerable results are highlighted enough. All the mentioned tests passed.